GradMap AI Coding Agent Execution Plan

Execution Rules

- [ ] Execute modules strictly in order: 1 → 2 → 3 → 4 → 5 → 6.
- [ ] Complete every checkbox before moving to the next major task.
- [ ] Do not implement any feature listed in the Deferred section during the Phase 1 build.
- [ ] Keep all secrets in environment variables or the deployment secret manager.
- [ ] Use TypeScript strict mode, server-side authorization, Drizzle migrations, and tested database queries.
- [ ] Record failed verification gates as blockers; do not silently continue.
- [ ] Add or update automated tests whenever a task changes authorization, persistence, billing, or destructive behavior.

Module 1: Environment & Secret Key Setup (`.env.local`)

1.1 Repository and Application Baseline

- [ ] Confirm the repository is a Next.js application using the App Router.
- [ ] Confirm TypeScript strict mode is enabled in `tsconfig.json`.
- [ ] Confirm Tailwind CSS is installed and configured.
- [ ] Confirm ESLint and the project formatter are configured.
- [ ] Add or confirm scripts for `dev`, `build`, `lint`, `typecheck`, `test`, and `test:e2e`.
- [ ] Create the required App Router route-group directories without implementing feature logic.
- [ ] Create `.env.example` containing names only and no real credentials.
- [ ] Add `.env.local` to `.gitignore`.
- [ ] Add a CI secret-scanning check for committed credentials.

Verification Gate 1.1

- [ ] Run `npm run lint` and confirm zero lint errors.
- [ ] Run `npm run typecheck` and confirm zero TypeScript errors.
- [ ] Run `git check-ignore .env.local` and confirm `.env.local` is ignored.
- [ ] Confirm no non-placeholder secret is committed to the repository.

1.2 Environment Variable Contract

- [ ] Add `NEXT_PUBLIC_APP_URL` to `.env.example`.
- [ ] Add `DATABASE_URL` to `.env.example`.
- [ ] Add `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` to `.env.example`.
- [ ] Add `CLERK_SECRET_KEY` to `.env.example`.
- [ ] Add `CLERK_WEBHOOK_SECRET` to `.env.example`.
- [ ] Add `STRIPE_SECRET_KEY` to `.env.example`.
- [ ] Add `STRIPE_WEBHOOK_SECRET` to `.env.example`.
- [ ] Add `STRIPE_MONTHLY_PRICE_ID` to `.env.example`.
- [ ] Add `STRIPE_YEARLY_PRICE_ID` to `.env.example`.
- [ ] Add `NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT` to `.env.example`.
- [ ] Add `NEXT_PUBLIC_IMAGEKIT_PUBLIC_KEY` to `.env.example`.
- [ ] Add `IMAGEKIT_PRIVATE_KEY` to `.env.example`.
- [ ] Add `GEMINI_API_KEY` to `.env.example` as a reserved Phase 2 variable only.
- [ ] Create a typed server-only environment configuration module.
- [ ] Make the configuration module fail fast when a required server variable is missing.
- [ ] Prevent private variables from being imported by client components.
- [ ] Add safe environment validation that reports variable names but never prints secret values.

Verification Gate 1.2

- [ ] Copy `.env.example` to `.env.local` with local placeholder values only.

- [ ] Start the app with `npm run dev` and confirm environment validation behaves as expected.
- [ ] Intentionally remove one required server variable and confirm startup reports the variable name without exposing a secret.
- [ ] Search client bundles/source imports and confirm `CLERK_SECRET_KEY`, `STRIPE_SECRET_KEY`, `IMAGEKIT_PRIVATE_KEY`, and `GEMINI_API_KEY` are server-only.

1.3 Local Service Connectivity

- [ ] Configure a development Neon PostgreSQL `DATABASE_URL`.
- [ ] Configure Clerk development keys.
- [ ] Configure Stripe test-mode keys and test Price IDs.
- [ ] Configure ImageKit development endpoint and public key.
- [ ] Keep Gemini unconfigured for Phase 1 unless the application boot process explicitly permits it to be absent.
- [ ] Document local setup commands in `README.md`.
- [ ] Document the required Stripe CLI webhook forwarding command in `README.md`.

Verification Gate 1.3

- [ ] Run the application locally and confirm the landing page loads.
- [ ] Confirm the application does not crash because `GEMINI_API_KEY` is absent in Phase 1.
- [ ] Confirm test-mode Stripe and development Clerk configuration are not production credentials.
- [ ] Run `npm run build` and confirm zero build errors.

Module 2: Database Schema & ORM Provisioning (Drizzle/PostgreSQL)

2.1 Drizzle and Neon Setup

- [ ] Install Drizzle ORM and the Neon PostgreSQL driver.

- [ ] Install the migration CLI used by the repository.
- [ ] Create `db/index.ts` with a server-only database client.
- [ ] Create `drizzle.config.ts` using `DATABASE_URL`.
- [ ] Create a `db/schema.ts` entry point.
- [ ] Create a migrations directory.
- [ ] Add a migration command to `package.json`.
- [ ] Add a migration inspection command to `package.json`.
- [ ] Configure a database seed command for local development.

Verification Gate 2.1

- [ ] Run the Drizzle configuration command and confirm the configuration loads.
- [ ] Connect to the development Neon database with a harmless connectivity query.
- [ ] Run `npm run typecheck` and confirm the database client has no TypeScript errors.

2.2 Shared PostgreSQL Conventions

- [ ] Enable `pgcrypto` or the PostgreSQL extension required for `gen_random_uuid()`.
- [ ] Define all primary keys as PostgreSQL `uuid` values.
- [ ] Set UUID defaults to `gen_random_uuid()`.
- [ ] Define timestamps as `timestamptz`.
- [ ] Set `created_at` defaults to `now()`.
- [ ] Add `updated_at` fields to mutable tables.
- [ ] Define foreign keys with `ON DELETE CASCADE` unless a task explicitly requires a different rule.
- [ ] Add server-side validation for all enum-like status and type values.
- [ ] Store timestamps in UTC.

Verification Gate 2.2

- [ ] Generate a migration and inspect it for UUID, timestamp, foreign-key, and cascade definitions.

- [ ] Confirm the generated migration contains no destructive operation that was not intended.
- [ ] Run the migration against the development database.
- [ ] Query the PostgreSQL catalog and confirm the expected extension is enabled.

2.3 Core Tables

- [ ] Create the `users` table with `id`, `clerk_user_id`, `email`, `display_name`, `timezone`, `created_at`, and `updated_at`.
- [ ] Add a unique constraint to `users.clerk_user_id`.
- [ ] Create the `workspaces` table with `id`, `owner_user_id`, `name`, `mode`, `created_at`, and `updated_at`.
- [ ] Restrict `workspaces.mode` to `career` or `masters`.
- [ ] Add the MVP unique owner constraint so one user has one workspace.
- [ ] Create the `applications` table with workspace, type, organization, title, location, source URL, deadline, status, intake year, notes, and timestamps.
- [ ] Restrict `applications.type` to `career` or `masters`.
- [ ] Create the `requirements` table with application relation, name, description, required flag, status, source note, and timestamps.
- [ ] Create the `tasks` table with application relation, title, description, due date, priority, status, completion timestamp, and timestamps.
- [ ] Restrict task priority to `low`, `medium`, or `high`.
- [ ] Restrict task status to `todo`, `in_progress`, or `done`.
- [ ] Create the `documents` table with workspace relation, uploader relation, name, document type, MIME type, file size, ImageKit file ID, URL, and timestamps.
- [ ] Create the `application_documents` join table with a composite primary key.
- [ ] Add cascading foreign keys from `application_documents` to both parent tables.
- [ ] Create the `subscriptions` table with workspace relation, Stripe customer ID, subscription ID, price ID, status, trial dates, billing period dates, cancellation flag, and timestamps.
- [ ] Add a unique workspace constraint to `subscriptions`.

- [ ] Create the `consents` table with user relation, consent type, policy version, granted flag, consent timestamp, withdrawal timestamp, and safe hash fields.
- [ ] Create the `stripe_events` table with Stripe event ID, event type, processed timestamp, payload hash, and creation timestamp.
- [ ] Add a unique constraint to `stripe_events.stripe_event_id`.

Verification Gate 2.3

- [ ] Generate the schema migration.
- [ ] Inspect every generated table, column type, constraint, and cascade rule.
- [ ] Apply the migration to the development database.
- [ ] Query each table from a database client and confirm it exists with the expected columns.
- [ ] Run `npm run typecheck` and confirm zero schema/type errors.

2.4 Indexes and Data Integrity

- [ ] Add a unique index on `users.clerk_user_id`.
- [ ] Add a unique index on `workspaces.owner_user_id` for MVP.
- [ ] Add a composite index on `applications.workspace_id, deadline_at`.
- [ ] Add a composite index on `applications.workspace_id, status`.
- [ ] Add an index on `requirements.application_id`.
- [ ] Add a composite index on `tasks.application_id, status, due_at`.
- [ ] Add a composite index on `documents.workspace_id, created_at`.
- [ ] Add unique indexes on Stripe customer and subscription IDs.
- [ ] Add a unique index on `stripe_events.stripe_event_id`.
- [ ] Add a database or service-level rule that application type matches workspace mode.
- [ ] Add validation preventing negative document file sizes.
- [ ] Add validation preventing `completed_at` from being set for an incomplete task unless the product explicitly permits it.

Verification Gate 2.4

- [ ] Run migration generation again and confirm no unintended schema drift.
- [ ] Execute database constraint tests for invalid modes, statuses, priorities, duplicate IDs, and mismatched application types.
- [ ] Confirm deadline list queries use the intended indexes with `EXPLAIN`.

2.5 Row Level Security

- [ ] Enable RLS on `users`.
- [ ] Enable RLS on `workspaces`.
- [ ] Enable RLS on `applications`.
- [ ] Enable RLS on `requirements`.
- [ ] Enable RLS on `tasks`.
- [ ] Enable RLS on `documents`.
- [ ] Enable RLS on `application_documents`.
- [ ] Enable RLS on `subscriptions`.
- [ ] Enable RLS on `consents`.
- [ ] Enable RLS on `stripe_events`.
- [ ] Define the trusted server-side mechanism for passing the authenticated Clerk identity into a database transaction.
- [ ] Allow users to read and update only their own profile row.
- [ ] Allow workspace owners to access only their own workspace.
- [ ] Restrict applications to the authenticated workspace owner.
- [ ] Restrict requirements and tasks through their parent application workspace.
- [ ] Restrict documents through their workspace ownership.
- [ ] Require both sides of an application-document link to belong to the same workspace.
- [ ] Allow ordinary authenticated users to read but not directly write subscription records.
- [ ] Restrict Stripe event access to the server-side webhook role.
- [ ] Preserve consent history while allowing consent withdrawal.

Verification Gate 2.5

- [ ] Create two test users and two test workspaces.
- [ ] Confirm User A cannot select User B's workspace, applications, requirements, tasks, documents, or subscription.
- [ ] Confirm User A cannot update or delete User B's rows by guessing UUIDs.
- [ ] Confirm webhook processing can write only through its controlled server-side path.
- [ ] Run the full RLS integration test suite and confirm all isolation tests pass.

2.6 Seed Data and Query Services

- [ ] Create seed data for one Career Mode workspace.
- [ ] Create seed data for one Master's Application Mode workspace.
- [ ] Add representative requirements and tasks to both seed workspaces.
- [ ] Create typed repository/service functions for users and workspaces.
- [ ] Create typed repository/service functions for applications.
- [ ] Create typed repository/service functions for requirements.
- [ ] Create typed repository/service functions for tasks.
- [ ] Create typed repository/service functions for documents and links.
- [ ] Create typed dashboard aggregate queries.
- [ ] Ensure every repository function requires an authenticated workspace context.

Verification Gate 2.6

- [ ] Run the seed command against a disposable development database.
- [ ] Confirm dashboard aggregate queries return correct totals, due-soon counts, and completion percentages.
- [ ] Run unit tests for every repository service.
- [ ] Run `npm run build` and confirm zero TypeScript or migration errors.

Module 3: Authentication & Protected Route Middleware

3.1 Clerk Integration

- [ ] Install and configure the Clerk Next.js SDK.
- [ ] Add Clerk environment variables to the server configuration.
- [ ] Add Clerk middleware for public and protected paths.
- [ ] Create the `/sign-in` route.
- [ ] Create the `/sign-up` route.
- [ ] Add sign-out behavior through Clerk.
- [ ] Create a server helper that returns the authenticated Clerk user ID.
- [ ] Create a server helper that resolves or creates the local `users` row.
- [ ] Do not store Clerk passwords, MFA secrets, or session tokens in PostgreSQL.

Verification Gate 3.1

- [ ] Register a test user through Clerk.
- [ ] Sign in and confirm the session survives a page refresh.
- [ ] Sign out and confirm protected pages redirect to sign-in.
- [ ] Confirm no secret Clerk key is present in the browser bundle.

3.2 Protected Route and API Authorization

- [ ] Protect `/onboarding` with authentication.
- [ ] Protect `/app/*` with authentication.
- [ ] Protect `/api/stripe/checkout` with authentication.
- [ ] Protect `/api/stripe/portal` with authentication.
- [ ] Protect `/api/upload/imagekit` with authentication.
- [ ] Keep `/api/stripe/webhook` publicly reachable for Stripe but protect it with signature verification.
- [ ] Reject unauthenticated mutation requests with a safe 401 response.
- [ ] Resolve workspace ownership on the server for every protected data request.
- [ ] Ignore client-supplied user ownership values.
- [ ] Reject a client-supplied workspace ID that does not belong to the authenticated user.

- [ ] Add a shared authorization helper for workspace-scoped entities.
- [ ] Add a subscription access helper that runs on the server.
- [ ] Return a safe 403 response for authenticated users without access to a protected mutation.

Verification Gate 3.2

- [ ] Test every protected page while signed out and confirm redirect behavior.
- [ ] Test every protected API route while signed out and confirm a 401 response.
- [ ] Test User A using User B's entity IDs and confirm a 403 or not-found response without data leakage.
- [ ] Run authorization integration tests and confirm all ownership checks pass.

3.3 First-Login Onboarding

- [ ] Add a server check for whether the authenticated user has a workspace.
- [ ] Redirect authenticated users without a workspace to `/onboarding`.
- [ ] Redirect users with a workspace to `/app`.
- [ ] Build the onboarding mode-selection screen.
- [ ] Add a Career Mode selection card.
- [ ] Add a Master's Application Mode selection card.
- [ ] Make the selected state visible through text and non-color visual feedback.
- [ ] Require the user to select exactly one mode.
- [ ] Add links to Terms of Service, Privacy Policy, and PICS.
- [ ] Add a required consent control for the privacy notice and Terms of Service.
- [ ] Add a separate optional marketing consent control.
- [ ] Record policy version and consent timestamps on submission.
- [ ] Create the personal workspace after valid onboarding submission.
- [ ] Prevent duplicate workspace creation on repeated submissions.
- [ ] Redirect the user to `/app` after successful workspace creation.

Verification Gate 3.3

- [ ] Complete onboarding as a new Career Mode user and confirm the workspace is persisted.
- [ ] Complete onboarding as a new Master's Application Mode user and confirm the workspace is persisted.
- [ ] Submit onboarding without required consent and confirm validation blocks the request.
- [ ] Submit onboarding twice and confirm only one workspace is created.
- [ ] Confirm consent rows contain policy version, granted state, and timestamps.

3.4 Privacy and Account Settings Foundation

- [ ] Create the public `/privacy` route.
- [ ] Create the public `/terms` route.
- [ ] Display the applicable PICS / privacy information disclaimer in onboarding.
- [ ] Create `/app/settings` for profile, mode, and privacy settings.
- [ ] Display the current consent records to the user.
- [ ] Add marketing consent withdrawal.
- [ ] Add a data deletion request entry point without deleting data silently.
- [ ] Add safe error and success feedback for consent changes.

Verification Gate 3.4

- [ ] Verify all privacy and Terms links work without authentication.
- [ ] Withdraw marketing consent and confirm `withdrawn_at` is recorded.
- [ ] Confirm required consent history is not overwritten.
- [ ] Confirm no sensitive document contents appear in page logs or error messages.

Module 4: Cloud Storage & Third-Party API Integrations

4.1 ImageKit Server Integration

- [ ] Install the ImageKit server SDK or compatible client.
- [ ] Create a server-only ImageKit configuration module.
- [ ] Create `/api/upload/imagekit` as an authenticated route.
- [ ] Validate the authenticated workspace before generating upload parameters.
- [ ] Generate short-lived ImageKit upload authentication parameters server-side.
- [ ] Never send `IMAGEKIT_PRIVATE_KEY` to the browser.
- [ ] Validate allowed MIME types server-side.
- [ ] Validate file extensions server-side.
- [ ] Enforce the MVP maximum file size.
- [ ] Accept only PDF, DOCX, PNG, and JPEG files unless product configuration explicitly changes the list.
- [ ] Return a safe upload response containing only required public/upload metadata.

Verification Gate 4.1

- [ ] Upload one valid PDF in development and confirm ImageKit returns a file ID and URL.
- [ ] Attempt an unsupported MIME type and confirm the server rejects it.
- [ ] Attempt an oversized file and confirm the server rejects it.
- [ ] Inspect the browser bundle and confirm the ImageKit private key is absent.

4.2 Document Persistence and Linking

- [ ] Build the document upload form.
- [ ] Save document name, type, MIME type, file size, ImageKit file ID, URL, and upload timestamp.
- [ ] Associate every document with the authenticated workspace.
- [ ] Associate every document with the authenticated uploader.
- [ ] Build the reusable document library query.
- [ ] Build the document type selector for CV, Resume, Personal Statement, Cover Letter, Recommendation Letter, Transcript, Portfolio, and Other.
- [ ] Link a document to an application through `application_documents`.

- [ ] Prevent links across workspaces.
- [ ] Unlink a document without deleting the original document.
- [ ] Delete the ImageKit object before deleting the local document record.
- [ ] Handle ImageKit deletion failure without silently losing local metadata.
- [ ] Add confirmation before document deletion.

Verification Gate 4.2

- [ ] Upload a valid document and confirm it appears in the document library.
- [ ] Link one document to two applications in the same workspace.
- [ ] Unlink one application and confirm the document remains available.
- [ ] Delete the document and confirm both the local record and ImageKit object are removed.
- [ ] Confirm a user cannot read, link, or delete another user's document.

4.3 Stripe Client and Checkout Boundary

- [ ] Install the Stripe server SDK.
- [ ] Create a server-only Stripe configuration module.
- [ ] Create typed billing status constants.
- [ ] Add monthly and yearly Stripe Price ID configuration.
- [ ] Create `/api/stripe/checkout` as an authenticated server route.
- [ ] Resolve the authenticated workspace before creating Checkout.
- [ ] Create or reuse a Stripe Customer for the workspace owner.
- [ ] Add workspace and user metadata to the Checkout Session.
- [ ] Use subscription mode for the Checkout Session.
- [ ] Configure the first-month-free trial on the server-side Checkout request.
- [ ] Configure success and cancel URLs from `NEXT_PUBLIC_APP_URL`.
- [ ] Return only the Stripe Checkout URL to the client.
- [ ] Do not accept arbitrary client-provided prices or Price IDs.
- [ ] Create `/api/stripe/portal` for the Customer Portal when enabled.

Verification Gate 4.3

- [ ] Start a Stripe test-mode Checkout session using the monthly option.
- [ ] Start a Stripe test-mode Checkout session using the yearly option.
- [ ] Confirm the server rejects an unconfigured or arbitrary Price ID.
- [ ] Confirm the Checkout Session contains the expected workspace metadata.
- [ ] Confirm no card number is received or stored by GradMap.

4.4 Phase 1 Third-Party Boundary Tests

- [ ] Add unit tests for environment validation.
- [ ] Add unit tests for ImageKit file validation.
- [ ] Add unit tests for workspace ownership before uploads.
- [ ] Add unit tests for Stripe Price ID selection.
- [ ] Add unit tests for server-only secret imports.
- [ ] Add integration tests for successful and rejected uploads.
- [ ] Add integration tests for Checkout session authorization.

Verification Gate 4.4

- [ ] Run the third-party integration test suite.
- [ ] Run `npm run lint` and confirm zero errors.
- [ ] Run `npm run typecheck` and confirm zero errors.
- [ ] Run `npm run build` and confirm zero build errors.

Module 5: Core UI Workspace & Canvas Components

5.1 Public Marketing Pages

- [ ] Build the public landing page at `/`.
- [ ] Add the GradMap value-proposition hero section.

- [ ] Explain Career Mode and Master's Application Mode in the supporting copy.
- [ ] Add the `Start your free month` primary CTA.
- [ ] Add the `See how it works` secondary CTA.
- [ ] Avoid employment, admission, visa, or success-rate guarantees.
- [ ] Build the public pricing page at `/pricing`.
- [ ] Add monthly and yearly pricing cards.
- [ ] Display the first-month-free trial terms clearly.
- [ ] Build the public Privacy Policy page.
- [ ] Build the public PICS/privacy information section.
- [ ] Build the public Terms of Service page.

Verification Gate 5.1

- [ ] Visit `/`, `/pricing`, `/privacy`, and `/terms` while signed out.
- [ ] Confirm all primary and secondary CTAs point to valid routes.
- [ ] Confirm pricing terms are visible without requiring a checkout action.
- [ ] Run a responsive check at mobile, tablet, and desktop widths.

5.2 Shared UI and Accessibility Foundation

- [ ] Create reusable layout, card, button, input, select, badge, table, dialog, toast, skeleton, and empty-state components.
- [ ] Define accessible color tokens with text contrast of at least 4.5:1.
- [ ] Define the keyboard focus style as a visible `2px solid` outline.
- [ ] Ensure focus styles are not clipped by containers.
- [ ] Add accessible labels to every form control.
- [ ] Add visible required-field indicators that do not rely on color alone.
- [ ] Add text alternatives for status colors and icons.
- [ ] Add keyboard support to dropdowns, dialogs, tabs, and popovers.
- [ ] Add Escape behavior to dialogs and popovers.
- [ ] Add loading, success, and error feedback to mutation components.

- [ ] Add responsive layout primitives for desktop, tablet, and mobile.
- [ ] Respect `prefers-reduced-motion` for non-essential animation.

Verification Gate 5.2

- [ ] Navigate the shared components using keyboard only.
- [ ] Run an automated accessibility scan on representative pages.
- [ ] Confirm focus rings are visible on buttons, links, inputs, dialogs, and table actions.
- [ ] Confirm error messages identify how the user can correct the problem.

5.3 Application Mode and Dashboard Shell

- [ ] Create the authenticated dashboard layout for `/app`.
- [ ] Add the workspace name to the header.
- [ ] Add the current mode badge to the header.
- [ ] Add the Clerk user menu.
- [ ] Add navigation links for Applications, Compare, Documents, Settings, and Billing.
- [ ] Add the dashboard summary cards for Total, In Progress, Due Soon, and Completed.
- [ ] Add the primary `New application` CTA.
- [ ] Add the deadline list/table.
- [ ] Add the upcoming tasks panel.
- [ ] Add the recent documents panel.
- [ ] Add the three-step empty state for a new workspace.
- [ ] Display the current mode name on the dashboard.
- [ ] Sort applications by deadline.
- [ ] Display each application's status.
- [ ] Display each application's completion percentage.
- [ ] Display the next incomplete task for each application.
- [ ] Display absolute dates alongside relative deadline text.

Verification Gate 5.3

- [ ] Sign in with a seeded Career Mode user and confirm the correct dashboard appears.
- [ ] Sign in with a seeded Master's Mode user and confirm the correct dashboard appears.
- [ ] Confirm dashboard totals match the database aggregate query.
- [ ] Confirm applications are sorted by deadline.
- [ ] Confirm empty state appears for a new workspace.
- [ ] Confirm loading, error, and success states are visible during dashboard data operations.

5.4 Application CRUD UI

- [ ] Build `/app/applications` application list page.
- [ ] Build `/app/applications/new` application creation form.
- [ ] Render Career Mode fields for company, role title, location, job URL, deadline, status, notes, and requirements.
- [ ] Render Master's Mode fields for university, programme name, location, programme URL, deadline, intake year, status, notes, and requirements.
- [ ] Render only statuses valid for the current mode.
- [ ] Validate required fields before submission.
- [ ] Submit creation through a server action or authenticated route.
- [ ] Display a success state after creation.
- [ ] Redirect to the new application detail page after creation.
- [ ] Build edit controls for an application.
- [ ] Submit edits through an authenticated server mutation.
- [ ] Add a delete action with a confirmation dialog.
- [ ] Prevent deletion of an application belonging to another workspace.
- [ ] Build `/app/applications/[id]` detail page.
- [ ] Display application overview, organization, title, deadline, and status.
- [ ] Display mode-specific detail labels.
- [ ] Add sections for Overview, Requirements, Tasks, Documents, and Notes.

Verification Gate 5.4

- [ ] Create, edit, view, and delete a Career Mode application.
- [ ] Create, edit, view, and delete a Master's Mode application.
- [ ] Confirm invalid mode-specific status values are rejected in both UI and server validation.
- [ ] Confirm a mismatched application type cannot be created through a direct request.
- [ ] Confirm destructive deletion requires confirmation and is keyboard accessible.

5.5 Requirements UI

- [ ] Add a requirement creation form on the application detail page.
- [ ] Add requirement name and description fields.
- [ ] Add the required/not-required control.
- [ ] Add requirement status selection.
- [ ] Add source note input.
- [ ] Display requirements in the application detail page.
- [ ] Add edit requirement behavior.
- [ ] Add delete requirement behavior.
- [ ] Render Career Mode examples such as education, skills, languages, portfolio, and assessments.
- [ ] Render Master's Mode examples such as GPA, subject background, language scores, GRE/GMAT, recommendation letters, Personal Statement, and portfolio.
- [ ] Display missing or incomplete requirements clearly.

Verification Gate 5.5

- [ ] Create and update requirements in both modes.
- [ ] Confirm requirement completion contributes to application completion percentage.
- [ ] Confirm source notes are displayed without unsafe HTML interpretation.
- [ ] Confirm users cannot access requirements through another user's application ID.

5.6 Tasks and Deadline UI

- [ ] Add a task creation form on the application detail page.

- [ ] Add task title and description fields.
- [ ] Add task due date input.
- [ ] Add low, medium, and high priority selection.
- [ ] Add todo, in-progress, and done status selection.
- [ ] Display tasks on the application detail page.
- [ ] Display upcoming tasks on the dashboard.
- [ ] Add task completion behavior.
- [ ] Set `completed_at` when a task changes to done.
- [ ] Clear or update `completed_at` when a done task becomes incomplete.
- [ ] Display prominent due-soon indicators.
- [ ] Do not implement email or push reminders in Phase 1.

Verification Gate 5.6

- [ ] Create a task and confirm it appears on the application and dashboard.
- [ ] Mark the task complete and confirm the completion timestamp is persisted.
- [ ] Confirm the next incomplete task changes correctly on the dashboard.
- [ ] Confirm overdue and due-soon indicators are understandable without color alone.
- [ ] Confirm task actions are keyboard accessible.

5.7 Requirements Comparison UI

- [ ] Build `/app/compare`.
- [ ] Load only applications belonging to the authenticated workspace.
- [ ] Require at least two selected applications.
- [ ] Prevent comparing Career Mode and Master's Mode applications together.
- [ ] Display requirement names in comparison rows.
- [ ] Display whether each requirement is required.
- [ ] Display current completion status for each application.
- [ ] Display source notes.
- [ ] Add a horizontally scrollable table for small screens.

- [ ] Add a stacked comparison-card fallback where appropriate.
- [ ] Use manually entered application data and source URLs only.
- [ ] Do not add website crawling.

Verification Gate 5.7

- [ ] Compare two Career Mode applications successfully.
- [ ] Compare two Master's Mode applications successfully.
- [ ] Confirm fewer than two selections shows an actionable validation message.
- [ ] Confirm cross-mode comparison is blocked.
- [ ] Confirm the comparison layout remains usable on a narrow viewport.

5.8 Documents UI

- [ ] Build `/app/documents`.
- [ ] Add document upload control.
- [ ] Add document type selection.
- [ ] Add upload progress feedback.
- [ ] Display document metadata in the library.
- [ ] Add application-document linking control.
- [ ] Add unlink behavior.
- [ ] Add delete confirmation.
- [ ] Display only workspace-owned documents.
- [ ] Provide a safe failure message for rejected uploads.

Verification Gate 5.8

- [ ] Upload one valid document through the UI.
- [ ] Link the document to an application.
- [ ] Confirm the document appears in both the library and application detail page.
- [ ] Unlink the document and confirm the original remains in the library.
- [ ] Delete the document and confirm the UI reflects deletion.

5.9 Settings and Billing UI Shell

- [ ] Add profile information to `/app/settings`.
- [ ] Add current mode display to `/app/settings`.
- [ ] Add privacy and consent controls to `/app/settings`.
- [ ] Build `/app/billing`.
- [ ] Display current plan.
- [ ] Display subscription status.
- [ ] Display trial-end time when applicable.
- [ ] Add monthly and yearly checkout buttons.
- [ ] Add Customer Portal entry when enabled.
- [ ] Build `/checkout/success` with webhook synchronization pending state.
- [ ] Build `/checkout/cancel` with a safe return path.
- [ ] Display a non-blocking trial-ending notice.

Verification Gate 5.9

- [ ] Confirm billing page renders for a new user with no active subscription.
- [ ] Confirm billing page renders `trialing`, `active`, `past_due`, and canceled states correctly.
- [ ] Confirm checkout success does not claim active status before webhook synchronization.
- [ ] Confirm billing actions are inaccessible to another workspace owner.

5.10 UI Quality Gate

- [ ] Run unit tests for form validation and status rendering.
- [ ] Run end-to-end tests for onboarding and first application creation.
- [ ] Run end-to-end tests for application, task, requirement, and document flows.
- [ ] Run an accessibility audit on landing, onboarding, dashboard, detail, compare, documents, and billing pages.
- [ ] Test keyboard-only navigation through the core journey.
- [ ] Test responsive layouts at mobile, tablet, and desktop widths.

- [ ] Test loading, empty, success, validation-error, authorization-error, and server-error states.

Verification Gate 5.10

- [ ] Run `npm run lint` and confirm zero errors.
- [ ] Run `npm run typecheck` and confirm zero errors.
- [ ] Run `npm test` and confirm all tests pass.
- [ ] Run `npm run test:e2e` and confirm the core journey passes.
- [ ] Run `npm run build` and confirm zero build errors.

Module 6: Billing Integration, Webhooks, & Pre-Launch Audits

6.1 Stripe Webhook Endpoint

- [ ] Create `/api/stripe/webhook` as a public server endpoint.
- [ ] Read the raw request body without JSON mutation before signature verification.
- [ ] Verify the Stripe signature with `STRIPE_WEBHOOK_SECRET`.
- [ ] Reject invalid signatures with a safe 400 response.
- [ ] Parse the verified Stripe event.
- [ ] Add idempotency lookup using `stripe_events.stripe_event_id`.
- [ ] Store the event type and payload hash without storing unnecessary payment payload data.
- [ ] Process `checkout.session.completed`.
- [ ] Process `customer.subscription.created`.
- [ ] Process `customer.subscription.updated`.
- [ ] Process `customer.subscription.deleted`.
- [ ] Process `invoice.paid`.
- [ ] Process `invoice.payment_failed`.

- [ ] Update the matching subscription using trusted Stripe customer/subscription IDs.
- [ ] Store trial start and end dates.
- [ ] Store current billing period dates.
- [ ] Store `cancel_at_period_end`.
- [ ] Treat the webhook-synchronized subscription as the billing source of truth.
- [ ] Return 2xx for verified unknown event types after safe logging.
- [ ] Avoid logging full payment payloads or card data.

Verification Gate 6.1

- [ ] Forward Stripe test events using the Stripe CLI.
- [ ] Confirm a valid event creates or updates one subscription row.
- [ ] Replay the same event and confirm no duplicate state change occurs.
- [ ] Send an invalid signature and confirm the event is rejected.
- [ ] Deliver events out of order and confirm final state remains correct.

6.2 Subscription Access and Lifecycle

- [ ] Implement server-side access for `trialing` subscriptions.
- [ ] Implement server-side access for `active` subscriptions.
- [ ] Define and implement the MVP grace behavior for `past_due`.
- [ ] Restrict create and edit mutations for `canceled` subscriptions.
- [ ] Restrict create and edit mutations for `incomplete_expired` subscriptions.
- [ ] Preserve all user data when access is restricted.
- [ ] Allow restricted users to view billing status.
- [ ] Allow restricted users to reactivate through Stripe Checkout or the Customer Portal.
- [ ] Prevent the frontend from overriding server-side subscription access.
- [ ] Add safe billing status error handling.

Verification Gate 6.2

- [ ] Test `trialing` access for dashboard, CRUD, upload, and task mutations.

- [ ] Test `active` access for dashboard, CRUD, upload, and task mutations.
- [ ] Test `past_due` according to the documented grace rule.
- [ ] Test `canceled` and `incomplete_expired` restrictions.
- [ ] Confirm restricted users retain read access to permitted billing/data views.

6.3 Billing End-to-End Audit

- [ ] Start a monthly trial Checkout session.
- [ ] Start a yearly trial Checkout session.
- [ ] Complete a test payment.
- [ ] Confirm `checkout.session.completed` reaches the webhook.
- [ ] Confirm subscription status becomes `trialing` or `active` according to Stripe response.
- [ ] Confirm the success page waits for server-side synchronization.
- [ ] Simulate invoice payment failure.
- [ ] Confirm the application reflects `past_due` or the configured failure state.
- [ ] Cancel at period end in Stripe test mode.
- [ ] Confirm the local cancellation flag updates.
- [ ] Confirm subscription deletion preserves workspace data.
- [ ] Confirm no card details are stored in the database or application logs.

Verification Gate 6.3

- [ ] Complete the full Stripe test-mode lifecycle from pricing page through webhook state.
- [ ] Confirm all Stripe event handlers are idempotent.
- [ ] Confirm billing state is correct after refreshing every checkout return page.
- [ ] Confirm a user cannot manipulate the browser to unlock restricted mutations.

6.4 Security Audit

- [ ] Run a repository secret scan.
- [ ] Search the repository for hardcoded Stripe secrets.
- [ ] Search the repository for hardcoded Clerk secrets.

- [ ] Search the repository for hardcoded ImageKit private keys.
- [ ] Search the repository for hardcoded database credentials.
- [ ] Confirm all provider signatures are verified.
- [ ] Confirm all database queries are parameterized through Drizzle.
- [ ] Confirm all entity reads and mutations perform ownership checks.
- [ ] Confirm upload MIME type, extension, and size validation runs server-side.
- [ ] Confirm unsafe HTML is escaped or rejected.
- [ ] Confirm error responses do not expose stack traces or secrets.
- [ ] Confirm rate limits exist for authentication-sensitive, upload, checkout, webhook, and mutation endpoints.
- [ ] Confirm cookies and sessions use secure production settings.
- [ ] Confirm logs redact PII and payment payloads.
- [ ] Confirm dependency vulnerabilities are reviewed.

Verification Gate 6.4

- [ ] Run the security test suite and confirm all tests pass.
- [ ] Run the dependency audit and record any accepted exceptions.
- [ ] Run a manual IDOR test using guessed application, task, document, and workspace UUIDs.
- [ ] Confirm all unauthorized requests fail without data leakage.

6.5 Privacy, GDPR, PDPO, and Data Lifecycle Audit

- [ ] Confirm the Privacy Policy explains collected data and purposes.
- [ ] Confirm the PICS explains required and voluntary data collection.
- [ ] Confirm third-party processors include Clerk, Stripe, ImageKit, and any enabled provider.
- [ ] Confirm cross-border processing language is included where applicable.
- [ ] Confirm consent is separated into required privacy/Terms consent and optional marketing consent.
- [ ] Confirm each consent record stores policy version and timestamp.

- [ ] Confirm consent withdrawal is available for optional marketing consent.
- [ ] Confirm the application does not request unnecessary sensitive identifiers.
- [ ] Confirm document contents are not written to logs or analytics.
- [ ] Confirm the account/data deletion request path is visible.
- [ ] Confirm deletion handling preserves only legally necessary records.
- [ ] Confirm retention decisions are documented.
- [ ] Confirm no Phase 2 AI document processing occurs in Phase 1.

Verification Gate 6.5

- [ ] Have Privacy Policy, PICS, consent, retention, and deletion flows reviewed by the appropriate privacy/legal owner.
- [ ] Execute a consent export/review test for a test user.
- [ ] Execute a marketing-consent withdrawal test.
- [ ] Confirm privacy-related routes and controls are accessible by keyboard.

6.6 WCAG 2.2 AA Audit

- [ ] Verify normal text contrast is at least 4.5:1.
- [ ] Verify large text contrast is at least 3:1.
- [ ] Verify all interactive controls expose a visible 2px solid focus outline.
- [ ] Verify focus outlines are not clipped.
- [ ] Verify every form control has an accessible label.
- [ ] Verify errors are announced or programmatically associated with controls.
- [ ] Verify status information is not communicated by color alone.
- [ ] Verify dialogs manage focus and support Escape.
- [ ] Verify tables have appropriate headers and responsive behavior.
- [ ] Verify images have meaningful alt text or empty alt when decorative.
- [ ] Verify all core journeys work with keyboard only.
- [ ] Verify pages remain usable at 200% zoom.
- [ ] Verify reduced-motion behavior.

Verification Gate 6.6

- [ ] Run axe or equivalent automated accessibility testing.
- [ ] Run Lighthouse accessibility testing.
- [ ] Perform a keyboard-only walkthrough of onboarding, dashboard, CRUD, compare, documents, and billing.
- [ ] Record and resolve all critical and serious accessibility findings.

6.7 Production Readiness and Release Gate

- [ ] Run all database migrations against a production-like staging database.
- [ ] Verify a migration rollback or recovery procedure.
- [ ] Configure production secrets in the deployment secret manager.
- [ ] Confirm production Stripe webhook URL and signing secret.
- [ ] Confirm production Clerk URLs and webhook configuration.
- [ ] Confirm production ImageKit configuration.
- [ ] Confirm Neon database backups/recovery expectations.
- [ ] Configure structured logging with PII redaction.
- [ ] Configure monitoring for 5xx errors, webhook failures, checkout failures, upload failures, database latency, and unauthorized access attempts.
- [ ] Create an incident runbook for secret rotation, webhook replay, access incidents, and data breaches.
- [ ] Confirm the application has no Phase 2 UI or server routes enabled accidentally.
- [ ] Confirm the application does not crawl third-party university or recruitment websites.
- [ ] Confirm the application does not submit applications on behalf of users.
- [ ] Confirm the application does not store third-party website credentials.
- [ ] Confirm no employment, admission, visa, or success-rate guarantee is presented.
- [ ] Confirm the first-time user can complete registration, mode selection, and first application creation within 10 minutes.
- [ ] Confirm users can create at least five applications and see them sorted by deadline.
- [ ] Confirm users can create and complete tasks.

- [ ] Confirm users can upload and reuse at least one document.

Final Verification Gate 6.7

- [ ] Run `npm run lint` and confirm zero errors.
- [ ] Run `npm run typecheck` and confirm zero errors.
- [ ] Run `npm test` and confirm all unit/integration tests pass.
- [ ] Run `npm run test:e2e` and confirm the core user journey passes.
- [ ] Run `npm run build` and confirm zero build errors.
- [ ] Confirm database migration status is clean.
- [ ] Confirm the secret scan is clean.
- [ ] Confirm accessibility audit has no unresolved critical or serious findings.
- [ ] Confirm Stripe test-mode lifecycle tests pass.
- [ ] Obtain product, engineering, security, and privacy sign-off before production deployment.

Deferred: Phase 2 and Post-Launch Expansion

Do not implement these items in the Phase 1 execution above unless the scope is explicitly re-approved.

- [ ] Enable both Career Mode and Master's Application Mode for one account.
- [ ] Add usage credit analytics.
- [ ] Add an AI usage meter.
- [ ] Add Gemini requirement extraction.
- [ ] Add Gemini document gap analysis.
- [ ] Add Gemini CV and Personal Statement suggestions.
- [ ] Add explicit consent and data-minimization flows for Gemini document processing.
- [ ] Add multi-tenant team seats.
- [ ] Add teammate invitations and shared workspaces.

- [ ] Add role-based permissions.
- [ ] Add career counselor or university advisor collaboration.
- [ ] Add email reminders.
- [ ] Add calendar integration.
- [ ] Add push notifications and deadline reminders.
- [ ] Add CSV/JSON import and export.
- [ ] Add public-URL requirement fetching only after robots, terms, and data-source compliance review.
- [ ] Add advanced filters and saved views.
- [ ] Add a kanban board.
- [ ] Add application conversion and preparation analytics.
- [ ] Add status-funnel analytics.
- [ ] Add document version control.
- [ ] Add document diffing.
- [ ] Add document expiry reminders.
- [ ] Add Stripe coupons and promo codes.
- [ ] Add regional pricing.
- [ ] Add invoicing workflows.
- [ ] Add an admin console.
- [ ] Add support tooling.
- [ ] Add an audit-log viewer.
- [ ] Add multilingual content.
- [ ] Add locale-specific privacy flows.

Explicitly Out of Scope Unless Separately Approved

- [ ] Automatically submit university or company applications on behalf of users.
- [ ] Store or process third-party website login credentials.
- [ ] Crawl restricted or login-required websites.
- [ ] Guarantee employment, admission, visa approval, or application success.

- [ ] Add interview video analysis.
- [ ] Add social matching.
- [ ] Add job-recommendation algorithms.
- [ ] Add administrator permissions or cross-workspace sharing to Phase 1.
- [ ] Generate formal documents that users can submit without reviewing them.